

GPU Kernel Programming: Refresher

Advanced GPU Short Course

Piotr Jarosik, Marcin Lewandowski, Piotr Karwat

us4us Ltd.

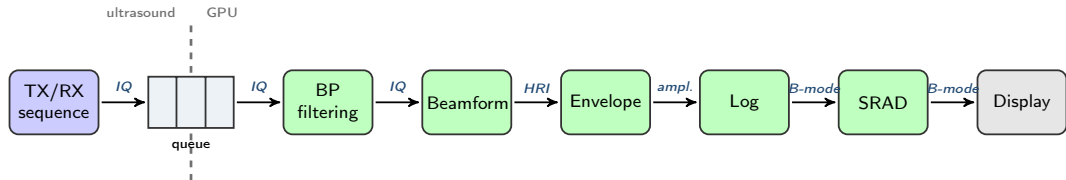
Agenda

Introduction

GPU Programming Basics

Summary

Example: Ultrasound B-mode image reconstruction



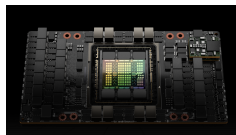
Architecture

Different GPU vendors, a single course

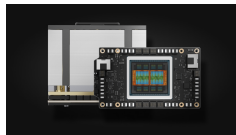
Objective

Implement **ultrasound imaging system**.

- multiple GPU vendors
- multiple software stacks (OpenCL, CUDA, HIP, SYCL, oneAPI, etc.)
- In this course:
 - ▶ **C++**, **CUDA** and **HIP** (+**SYCL**)
 - ▶ evaluation:
 - NVIDIA: **H100**
 - AMD: **MI350X**



NVIDIA H100¹



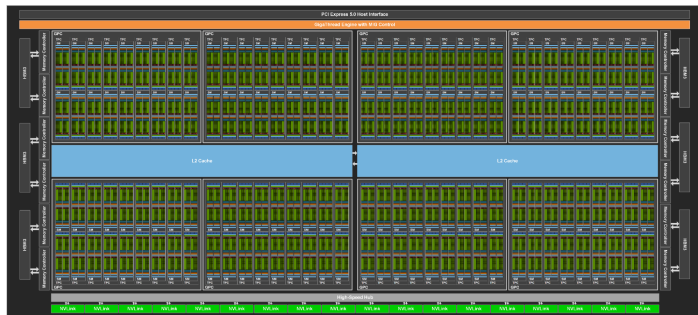
AMD MI350X²

¹<https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>

²<https://www.amd.com/en/products/accelerators/instinct/mi350/mi350x.html>

GPU architecture

NVIDIA GPU Architecture (Hopper, H100)

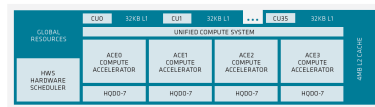
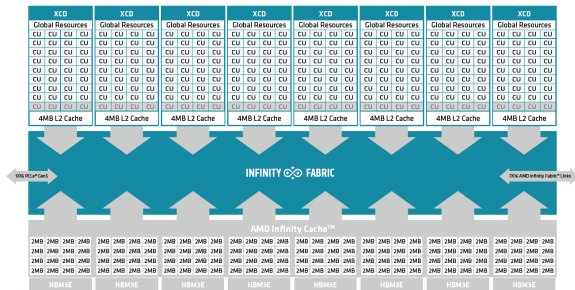


left: full H100 GPU with 144 SMs, right: GH100 streaming multiprocessor).¹

¹ NVIDIA, *NVIDIA Hopper Architecture In-Depth*, <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>

GPU architecture

AMD CDNA GPU Architecture (CDNA 4, MI350X)



left: MI350 series GPU, right: CDNA 4 accelerator complex die (XCD).²

² AMD, *AMD Instinct MI350 Series microarchitecture*, <https://rocm.docs.amd.com/en/latest/reference/gpu-arch/mi350.html>

Terminology: GPU architecture

	NVIDIA	AMD
lane / scalar core	CUDA core	stream processor (SIMD lane)
processor	Streaming Multiprocessor (SM)	Compute Unit (CU)
matrix processor	Tensor Core	Matrix Core
work distributor	GigaThread Engine	Command Processor, ACE, HWS
cluster of processors	Graphics Processing Cluster (GPC)	Accelerator Complex Die (XCD)

We will use the **CUDA** terminology and syntax in this course.

Programming model

Heterogeneous programming

Host

The host computer (with CPU and RAM): runs the process, allocates device memory, orders GPU $\leftrightarrow$ RAM transfers, **invokes** GPU kernels.



host: CPU + RAM

Device

GPU: executes **kernels**, which operate on *device* memory only.



device: GPU + VRAM

Photos: IBM PC – Rama & Musée Bolo, CC BY-SA 2.0 FR; GeForce RTX 5090 – ZMASLO, CC BY 3.0; Wikimedia Commons.

Heterogeneous programming

Kernel

A function invoked by the host, executed on GPU by multiple threads in parallel. Typically invoked **asynchronously**.

Data transfers

Copy host RAM $\leftrightarrow$ device RAM:

- HtoD: host RAM $\rightarrow$ device RAM
- DtoH: host RAM $\leftarrow$ device RAM

```
--global__  
void log(float *out, const float *in, float ref, int n){  
    /*...*/  
}
```



Heterogeneous programming

```
float *in, *out;
size_t bytes = n*sizeof(float);

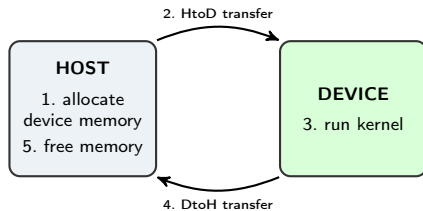
// 1. allocate device memory
cudaMalloc(&in, bytes);
cudaMalloc(&out, bytes);

// 2. HtoD transfer
cudaMemcpy(in, h_in, bytes, cudaMemcpyHostToDevice);

// 3. run the kernel (asynchronous)
log_compression<<<grid, block>>>(out, in, ref, n);

// 4. DtoH transfer (waits for the kernel)
cudaMemcpy(h_out, out, bytes, cudaMemcpyDeviceToHost);

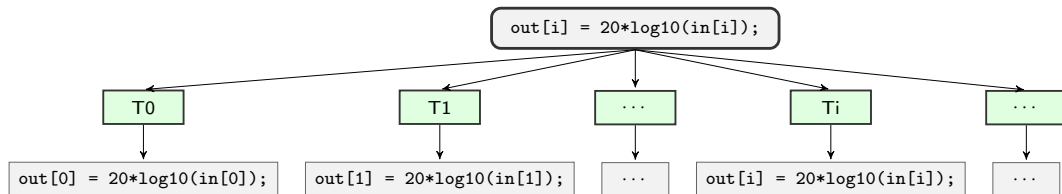
// 5. free device memory
cudaFree(in);
cudaFree(out);
```



SIMT: threads and warps

Warp

A collection of threads executed by Streaming Multiprocessor. All threads execute a **single instruction at a time**, each on its own data.



SIMT: types of instructions

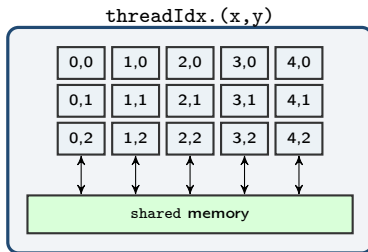
- **arithmetic** – kernel spends most time computing: **compute-bound**
- **load / store** – kernel spends most time loading data: **I/O-bound**
- **control flow** (if, for ...) – threads may take different branches: on **branch divergence** they are executed one after another

Thread blocks

Block

A logical collection of threads (up to 1024) executed by a single SM.

- Is never split and never migrates between SMs.
- thread index: `threadIdx.{x,y,z}` the thread's position in the block, x varies fastest.
- block size: `blockDim.{x,y,z}`.
- Threads can communicate through **shared memory** (on-chip, low latency, common to the whole block) and a **synchronization barrier** may be required (`__syncthreads()`).

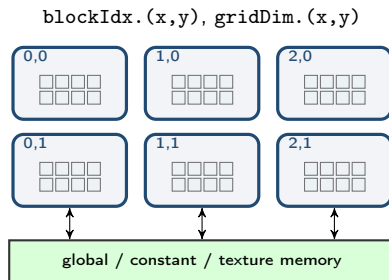


Grid of blocks

Grid

A logical collection of thread blocks that executes a single kernel launch on one GPU.

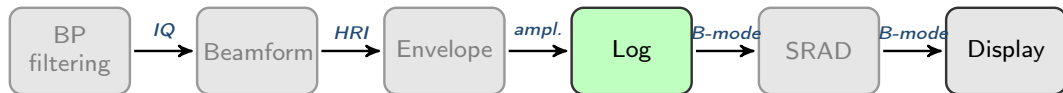
- Purpose: distribute the whole task over the blocks (each block: up to 1024 threads).
- Blocks are **independent**: they are scheduled on the available SMs in any order, so the same kernel scales from a small GPU to a large one.
- block index: `blockIdx.{x,y,z}` the block's position in the grid, x varies fastest.
- grid size: `gridDim.{x,y,z}`.
- No barrier between blocks within one kernel launch – the end of the kernel is the barrier.



Grid, block and warp sizes

Architecture	warp size	max threads / block	max block dim. (x, y, z)	max grid dim (x, y, z)
NVIDIA H100 (Hopper)	32	1024	1024, 1024, 64	$2^{31} - 1$, 65535, 65535
AMD MI350X (CDNA 4)	64	1024	1024, 1024, 1024	$2^{32} - 1$, 65535, 65535
AMD Radeon RX 7000/9000 (RDNA 3/4)	32	1024	1024, 1024, 1024	$2^{32} - 1$, 65535, 65535

Example: log-compression



$$y[i] = 20 \log_{10} (x[i]/x_{ref})$$

Example: log-compression

What job each thread should perform?

- Typically: **one thread per output value**, here: one thread per output pixel.
- Grid size: $\lceil n/\text{block} \rceil$
- The number of threads in the grid is a **multiple of the block size**, the last block may be only partially used – needs a **guard** against going outside the data.

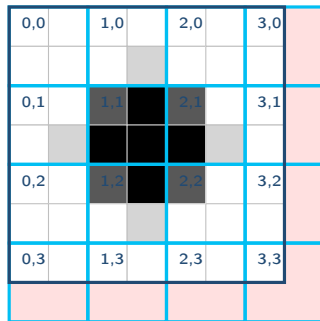


image: 7×7 pixels, block: 2×2 threads, grid: 4×4 blocks
red: threads outside the image $\Rightarrow$ guard

Example: log-compression

GPU kernel:

```
__global__  
void log_compression(float *out, const float *in,  
    float ref, int n) {  
  
    int i = blockIdx.x*blockDim.x + threadIdx.x;  
  
    if(i >= n) {  
        // do not reach outside input image  
        return;  
    }  
    out[i] = 20.0f*log10f(in[i]/ref);  
}
```

host code:

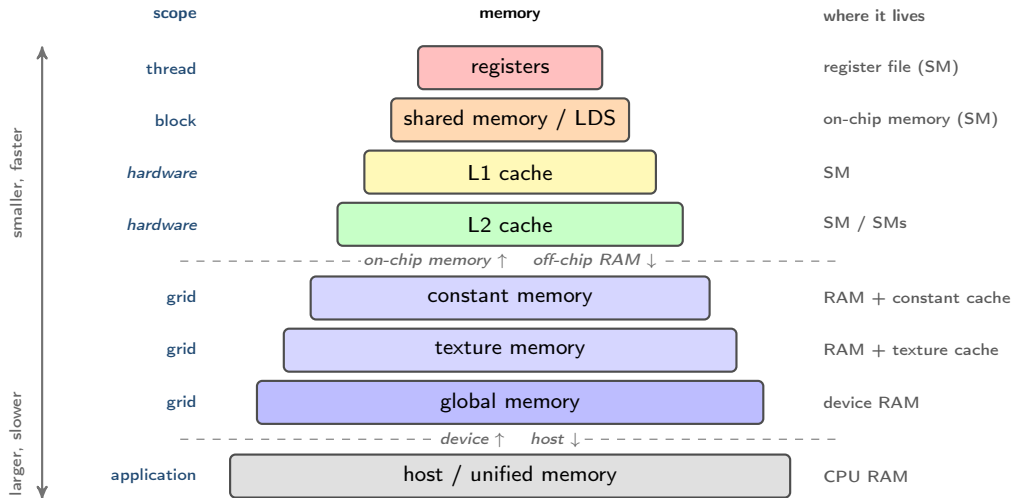
```
int n = nx*nz; // number of pixels  
int block = 256;  
int grid = (n + block - 1)/block;  
  
log_compression<<<grid, block>>>(out, in,  
    ref, n);
```

Terminology: programming model

CUDA	HIP (AMD)	SYCL
thread	thread (work-item)	work-item
warp	warp / wavefront	sub-group
thread block	block (workgroup)	work-group
grid	grid	ND-range

Memory Model

Memory hierarchy



Memory hierarchy: how much, and how fast

	NVIDIA H100 SXM5 (Hopper)		AMD MI350X (CDNA 4)	
	size	bandwidth	size	bandwidth
registers	256 KB per SM	register speed	512 KB per CU [†]	register speed
shared memory / LDS	up to 227 KB per block	128 B/clock per SM, ≈ 30 TB/s	160 KB per CU [†]	128 B/clock per CU
L1	shares 256 KB with shared memory	–	32 KB per CU	–
L2 cache	50 MB	–	32 MB (4 MB per XCD)	–
constant memory	64 KB	one broadcast per clock	no separate space	via the scalar cache
texture memory	a view of device memory	as global	a view of device memory	as global
global memory	80 GB HBM3	3.35 TB/s	288 GB HBM3E	8 TB/s
host memory	PCIe Gen5 x16	64 GB/s	PCIe Gen5 x16	64 GB/s

[†] to be confirmed on the target (hipGetDeviceProperties / rocminfo). “–”: no vendor figure published – measure it if you need it.

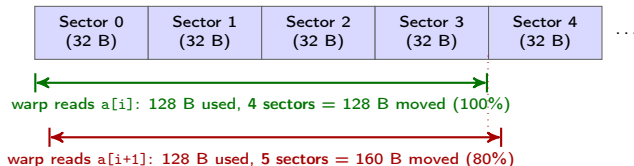
Shared memory: 32 banks $\times$ 4 B per clock; the aggregate assumes 132 SMs at 1.755 GHz.

Host & unified memory

- The host CPU RAM.
- NVIDIA GPUs provide three different access methods to host RAM:
 - ▶ **unified memory** (`cudaMallocManaged`): one virtual address space, data migrated between host and device on e.g. page fault.
 - ▶ **pinned memory** (`cudaMallocHost`): cannot be swapped out to disk (at the cost of OS pinned pages).
 - ▶ **pageable memory** (`malloc`): datatransfer is staged through driver's internal pinned memory (one more HtoH transfer: your buffer → driver's internal pinned buffer)
- Scope:
 - ▶ host PC (just dereference a pointer)
 - ▶ all GPU threads (directly via memory pointer: unified or pinned+mapped memory)

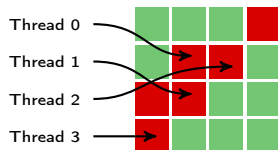
Global memory

- **Device DRAM.**
- Scope:
 - ▶ all GPU threads
 - memory ptr, e.g. `void beamform(float* out, float *in);`
 - ▶ host: malloc, free, DtoH and HtoD transfers
- Physically accessible through L2 (and through L1 on NVIDIA $\geq$ Volta)
- Atomic load / store is 32-byte sector – DRAM burst atom, GDDR6 32-bit channel with burst length 8.



Texture memory

- **A read-only view of texture unit memory (cache) accessed through the GPU texture units.**
- Optimized for global memory access patterns with spatial proximity.
- Provides basic interpolation and extrapolation.
- Scope:
 - ▶ all GPU threads, *read-only*, via texture object
 - ▶ host: `cudaArray`, `cudaMallocArray`, `cudaMemcpy2DToArray`, etc.



Constant memory

- Resides in **device memory** and is read by an SM through the **constant cache** (much cheaper than global memory read).
- **Read-only** for the kernel: the value of `__constant__` symbol must be set by the host before launching any kernel.
- Much smaller than global memory: **64 KB** – the same for every compute capability to date (CC 1.x – 12.x), with an 8 KB constant cache working set per SM.
- Scope:
 - ▶ all GPU threads, *read-only*, declared `__constant__` at file scope (size fixed at compile time)
 - ▶ host: `cudaMemcpyToSymbol` / `cudaMemcpyFromSymbol`
- Optimized for data **broadcast**: threads of a warp reading the *same* address are served in a single step (cheap). Threads reading *different* addresses are **serialized**, one access per unique address.

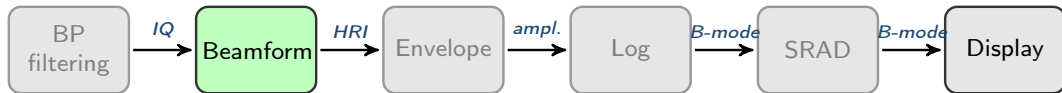
Shared memory

- A **low-latency memory near processor core** (close to L1 cache).
- Typically used as a **scratchpad** memory (software-managed cache).
- The access time is **much shorter** than to global memory, and the size is **much smaller**.
- Scope:
 - ▶ **one block** of threads, for the lifetime of that block: `__shared__` (static) or `extern __shared__` + a launch parameter (dynamic)
 - ▶ *not* accessible by the host.
- The classic use is **tiling** (see HPC patterns, Stencil).
- Divided into **32 banks** of 4-byte words: threads reading different bytes in the same bank are serialized (*bank conflict*), otherwise a single value broadcasted to threads reading the same bytes.

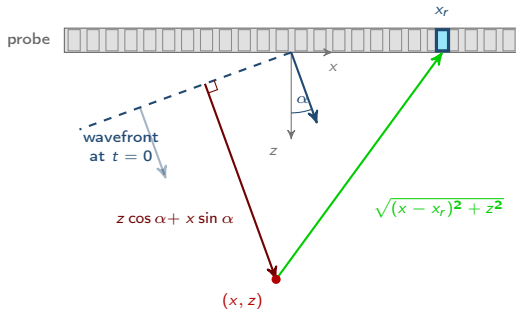
Registers

- The fast on-chip memory, allocated by the compiler out of the SM's register file (e.g. NVIDIA 64K 32-bit registers per SM).
- Declared as **ordinary local variable** of the kernel.
- Scope:
 - ▶ one thread only (but also see **warp shuffles**)
 - ▶ not accessible the host, and not to other threads
- **Register-hungry** kernels reduce GPU occupancy. The register file size limits the number of threads that can occupy SM: $\text{registers/thread} \times \text{threads/SM} \leq \text{register file size}$.
- **Spilling**: if thread needs more registers than it is allowed (*or accesses a local array using non-constant index*): the local variable is **spilled** to *local memory*, which lives in **global memory**.
- NVIDIA: threads of the same warp can exchange registers directly with **warp shuffles** (`__shfl_sync`, etc.)

Example: beamforming



Example: beamforming



$$\tau_r(\alpha, x, z) = \frac{\overbrace{z \cos \alpha + x \sin \alpha}^{\text{transmit}} + \overbrace{\sqrt{(x - x_r)^2 + z^2}}^{\text{receive}}}{c} + t_0 \quad s(\alpha, x, z) = \sum_{r=1}^{N_{rx}} i_{q_r}(\tau_r) e^{j\omega \tau_r}$$

Example: beamforming – baseline (global memory)

```
__global__ void beamform(float2 *out, const float2 *iq, const float *xElem,
                        const float *alpha, const float *zPix, const float *XPix, ...) {

    // One thread per output Low Resolution Image (LRI) pixel.

    int z = blockIdx.x*blockDim.x + threadIdx.x;
    int x = blockIdx.y*blockDim.y + threadIdx.y;
    int tx = blockIdx.z;
    /** guards, ... */
    float txDistance = z*cosf(alpha[tx]) + x*sinf(alpha[tx]);
    float2 value = make_float2(0.0f, 0.0f);

    for(int iRx = 0; iRx < nRx; ++iRx) {
        float rxDistance = hypotf(x - xElem[iRx], z);
        float time = (txDistance + rxDistance)/sos;
        // fractional sample index
        float iSamp = time*fs;
        /** ... */
        // linear interpolation, by hand: two loads + a lerp
        int s = (int)iSamp; float w = iSamp - s;
        const float2 *ch = iq + (iTx*nRx + iRx)*nSamp;
        float2 a = ch[s], b = ch[s+1];
        float2 samp = make_float2(a.x*(1-w) + b.x*w, a.y*(1-w) + b.y*w);
        // IQ correction
        float sn, cs; __sincosf(omega*time, &sn, &cs);
        value.x += samp.x*cs - samp.y*sn;
        value.y += samp.x*sn + samp.y*cs;
    }
    out[z + x*nZPix + iTx*nZPix*nXPix] = pix;
```

Example: beamforming – texture memory

```
// host: IQ data as a 2D texture, one row per (tx, rx) channel, linear filtering
cudaCreateTextureObject(&iqTex, &res, &tex, nullptr);

__global__ void beamform(float2 *out, cudaTextureObject_t iqTex, const float *xElem, ...) {
    // ...
    for(int iRx = 0; iRx < nRx; ++iRx) {
        // ...
        float iSamp = time*fs;
        // ...
        float row = iTx*nRx + iRx;
        float2 samp = tex2D<float2>(iqTex, iSamp + 0.5f, row + 0.5f);
        // ...
    }
    // ...
}
```

- Replace the pointer to the IQ data with a **texture object**.
- The manually implemented interpolation becomes **one tex2D fetch**.
- Texture's 2D locality mitigates the scattered iSamp.
- Free edge clamping implementation.

Example: beamforming – constant memory

```
// file scope, size known at compile time
__constant__ float xElementConst[1024];
__constant__ float alphaConst[64];

__global__ void beamform(float2 *out, cudaTextureObject_t iqTex, ...) { // no xElement, alpha
    // ...
    float txDist = z*cosf(alphaConst[iTx]) + x*sinf(alphaConst[iTx]);
    // ...
    for(int iRx = 0; iRx < nRx; ++iRx) {
        float rxDistance = hypotf(x - xElementConst[iRx], z);
        // ...
    }
    // ...
}
```

host:

```
cudaMemcpyToSymbol(xElemConst, h_xElem, nElem*sizeof(float));
```

- The probe geometry is the same for every thread – we can put it into a constant memory.
- At a given for loop iteration, all 32 threads read `xElemConst[iRx]` $\Rightarrow$ one read **broadcast** to all threads.

Terminology: memory

CUDA	HIP (AMD)	SYCL
registers	scalar/vector general purpose registers	private memory
local memory	private memory	private memory
shared memory (<code>__shared__</code>)	Local Data Share (LDS), (<code>__shared__</code>)	local memory (<code>local_accessor</code>)
global memory	global memory	global memory
constant memory (<code>__constant__</code>)	constant memory (<code>__constant__</code>)	–

References

References: book and CUDA documentation

Book

- W.-m. W. Hwu, D. B. Kirk, I. El Hajj, *Programming Massively Parallel Processors: A Hands-on Approach*, 4th ed., Morgan Kaufmann, 2022.

Closest to this deck: ch. 2–3 (data-parallel computing, multidimensional grids), ch. 4 (compute architecture and scheduling), ch. 5 (memory architecture and data locality), ch. 6 (performance considerations).

NVIDIA CUDA

- *CUDA C++ Programming Guide* – <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
compute capabilities table (per-SM limits), device memory accesses, shared memory
- *CUDA C++ Best Practices Guide* – <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
coalesced access to global memory
- *Volta Tuning Guide* – <https://docs.nvidia.com/cuda/volta-tuning-guide/>
L1 caching of global loads is on by default since Volta

References: AMD, SYCL and the architecture figures

AMD HIP / ROCm, and SYCL

- *HIP documentation* – [https://rocm.docs.amd.com/projects/HIP/en/latest/hardware implementation; the hardware features table \(warp size, block and grid limits\)](https://rocm.docs.amd.com/projects/HIP/en/latest/hardware implementation; the hardware features table (warp size, block and grid limits))
- *AMD Instinct MI350 Series microarchitecture* – <https://rocm.docs.amd.com/en/latest/reference/gpu-arch/mi350.html>
- *SYCL 2020 Specification* – <https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html>

Architecture figures and specifications

- NVIDIA, *NVIDIA Hopper Architecture In-Depth* – <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>
the full-GH100 and SM diagrams
- NVIDIA H100 architecture whitepaper and SXM5 datasheet – <https://resources.nvidia.com/en-us-tensor-core>,
<https://www.nvidia.com/en-us/data-center/h100/>
- AMD Instinct MI350X product page – <https://www.amd.com/en/products/accelerators/instinct/mi350/mi350x.html>

References: course materials and code

Course materials

- *Lab4US GPU short course* (rev. 1, CuPy notebooks) – <https://github.com/Lab4US/gpu-short-course>
exercises/cupy/2_CUDA_memory_model: the shared and constant memory text; the convolve1d tiling solutions
- *CUDA Delay-and-Sum*, IUS 2024 short course – <https://github.com/Lab4US/gpu-short-course/tree/main/slides/ius-2024>
the texture-memory access-pattern figure, the interpolation examples

Code

- **arrus** – <https://github.com/us4useu/arrus>
api/python/arrus/utils/iq_raw_2_lri.cu: the RX beamforming kernel the example is reduced from
- the benchmarks of this course: examples/ in this repository
01_log_compression (one thread per output pixel), 05_beamform_pw (global vs texture vs constant memory). Each builds for CUDA and HIP and prints the table the slides quote.